

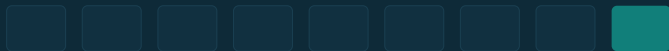


STREAM PROCESSING AVEC APACHE KAFKA

Interactive Queries & Tests

Interroger l'état, tester la topologie

ESGI · 5^e année · Data Engineering · 2025–2026



SÉANCE 14

Objectifs

- **Interroger l'état en direct**

Lire un state store sans repasser par un topic : les *Interactive Queries*.

- **Exposer l'état**

Une petite API REST par-dessus le store, et l'aspect *distribué*.

- **Tester sans cluster**

`TopologyTestDriver` : piper des entrées, vérifier sorties et stores.

- **Préparer le projet**

Deux briques directement réutilisables — et valorisées au barème.

SÉANCE 14

Au programme

01

Interactive Queries

interroger & exposer l'état

02

Tests

TopologyTestDriver

01

- PARTIE 1

Interactive Queries

L'état est déjà là



- Une agrégation maintient déjà un `KTable` dans un **state store** local. Le republier dans un topic juste pour le lire est un détour.
- Les **Interactive Queries** lisent ce store **directement**, en lecture seule — idéal pour exposer une API.

Interroger un KeyValueStore

IQ v1 : accéder au store en lecture seule

```
ReadOnlyKeyValueStore<String, Long> store =  
    streams.store(StoreQueryParameters.fromNameAndType(  
        "orders-count-by-currency-store",  
        QueryableStoreTypes.keyValueStore()));  
  
Long n = store.get("EUR");           // le compteur courant  
try (var it = store.all()) { ... }   // parcourir toutes les cles
```

IQ v2 (moderne) : `streams.query(...)` avec `KeyQuery`, `RangeQuery`,
`WindowKeyQuery...`

Une petite API REST (JDK)

Un endpoint `/count/DEVISE` qui interroge le store

```
HttpServer http = HttpServer.create(
    new InetSocketAddress(7070), 0);
http.createContext("/count/", ex -> {
    String ccy = ex.getRequestURI().getPath()
        .substring(7).toUpperCase();
    Long n = store.get(ccy); // Interactive Query
    byte[] body = (ccy + " = " + n).getBytes();
    ex.sendResponseHeaders(200, body.length);
    ex.getResponseBody().write(body); ex.close();
});
http.start();
```

Gérer `InvalidStateStoreException` (rebalance) : renvoyer un 503 le temps que le store redevienne prêt.

Quand l'état est réparti

- Le store est **partitionné** entre les instances : une instance ne détient que sa part de l'état. La clé cherchée peut vivre **ailleurs**.
- `streams.queryMetadataForKey(store, cle, serializer)` indique **quelle instance** héberge la clé → on lui **transfère** la requête (RPC).

■ Local

La clé est ici : on lit le store local directement.

■ Distant

La clé est ailleurs : on relaie vers l'instance active (`application.server`).

Interroger l'état par HTTP

Lancer, alimenter, interroger l API

```
mvn -q compile exec:java \  
  -Dexec.mainClass=fr.esgi.kafka.tp14.CountApiApp &  
echo 'o-1;19.9;eur' | kafka-console-producer.sh \  
  --bootstrap-server localhost:29092 --topic orders  
curl http://localhost:7070/count/EUR      # -> EUR = 1
```

■ Direct

Aucune relecture de topic : la réponse vient du store.

■ Vivant

Chaque commande produite fait monter le compteur renvoyé.

02

- PARTIE 2

Tests — TopologyTest- Driver

Tester sans cluster

- `TopologyTestDriver` exécute une topologie **sans broker** : ni Docker, ni réseau. On *pipe* des entrées et on lit les sorties **immédiatement**.
- Traitement **synchrone** et **déterministe** : on contrôle même les *horodatages* — parfait pour tester le fenêtrage et le retard.

■ Rapide

Des millisecondes par test : idéal en intégration continue.

■ Fiable

Pas de dépendance externe, résultats reproductibles.

Piper, lire, vérifier

TopologyTestDriver + TestInputTopic + TestOutputTopic

```
driver = new TopologyTestDriver(CountApiApp.buildTopology(), props);
in  = driver.createInputTopic("orders",
    new StringSerializer(), new StringSerializer());
out = driver.createOutputTopic("count-by-currency",
    new StringDeserializer(), new StringDeserializer());

in.pipeInput("k", "o-1;19.9;eur");
assertEquals("EUR", out.readKeyValue().key);    // JUnit 5
```

Extraire la topologie dans une méthode `buildTopology()` la rend testable sans démarrer Kafka.

Vérifier le store et l'évent time

Horodatages contrôles + accès direct au store

```
in.pipeInput("k", "a;1;eur", Instant.ofEpochMilli(1000));  
in.pipeInput("k", "b;1;eur", Instant.ofEpochMilli(2000));  
  
KeyValueStore<String, Long> store =  
    driver.getKeyValueStore("orders-count-by-currency-store");  
  
assertEquals(2L, store.get("EUR"));    // état agrégé vérifié
```

`pipeInput(k, v, Instant)` fixe l'évent time ; `getKeyValueStore` inspecte l'agrégat directement.

Lancer la suite de tests

Executer les tests, sans aucun cluster

```
mvn -q test
# [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
# [INFO] BUILD SUCCESS
```

■ Sans Docker

Les tests tournent seuls : aucun broker à démarrer.

■ Instantané

La suite s'exécute en quelques millisecondes.

OUVERTURE

Place au projet

- Fin de l'apprentissage guidé : vous avez de quoi bâtir un **pipeline de bout en bout** — ingestion, transformations, état, fenêtres, jointures, requêtes et tests.
- Le **projet** met tout en œuvre. La note suit un **barème progressif** : une base qui tourne, puis des *bonus* défendus à l'oral.

■ Acquis

Interroger l'état (IQ) et prouver la logique (tests).

Bonus projet

Tests +4, IQ +3, Avro, sécurité, EOS...

À retenir

- ✓ Les **Interactive Queries** lisent un state store **directement**, sans repasser par un topic.
- ✓ `streams.store(StoreQueryParameters...)` donne une vue lecture seule ; on l'expose via une API.
- ✓ En distribué : `queryMetadataForKey` localise la clé, puis **RPC** vers l'instance active.
- ✓ `TopologyTestDriver` teste la topologie **sans cluster**, de façon synchrone et déterministe.
- ✓ `pipeInput` (avec `Instant`) et `getKeyValueStore` vérifient sorties, état et fenêtres.



LA SUITE — LE PROJET

Un pipeline de bout en bout

Ingestion, traitement, état, requêtes et tests sur une vraie source — évalué au barème progressif, défendu à l'oral.